Lab 2: Build Your VPC and Launch a Web Server

Objective

The objective of this lab was to gain hands-on experience with Amazon Virtual Private Cloud (VPC) by creating a custom virtual network, configuring subnets across Availability Zones, setting up routing and internet access components, creating a security group, and launching an EC2 instance running a web server. I learned how to build isolated yet connected network environments in AWS while applying key concepts such as public/private subnets, high availability, and security group rules. **Key AWS services and concepts covered:**

- Amazon Virtual Private Cloud (VPC)
- Subnets (public and private)
- Internet Gateway and NAT Gateway
- Route Tables and subnet associations
- Security Groups
- EC2 instance launch with user data script
- High Availability through multiple Availability Zones

Scenario

In this lab, I built a custom VPC infrastructure suitable for hosting a web application. The architecture included a VPC with public and private subnets in two Availability Zones, an Internet Gateway for public access, a NAT Gateway for private subnet outbound connectivity, appropriate route tables, a security group allowing HTTP traffic, and an EC2 instance in a public subnet configured as a web server using a user data script. The goal was to ensure the web server was publicly accessible while understanding how network isolation and internet connectivity function in AWS.

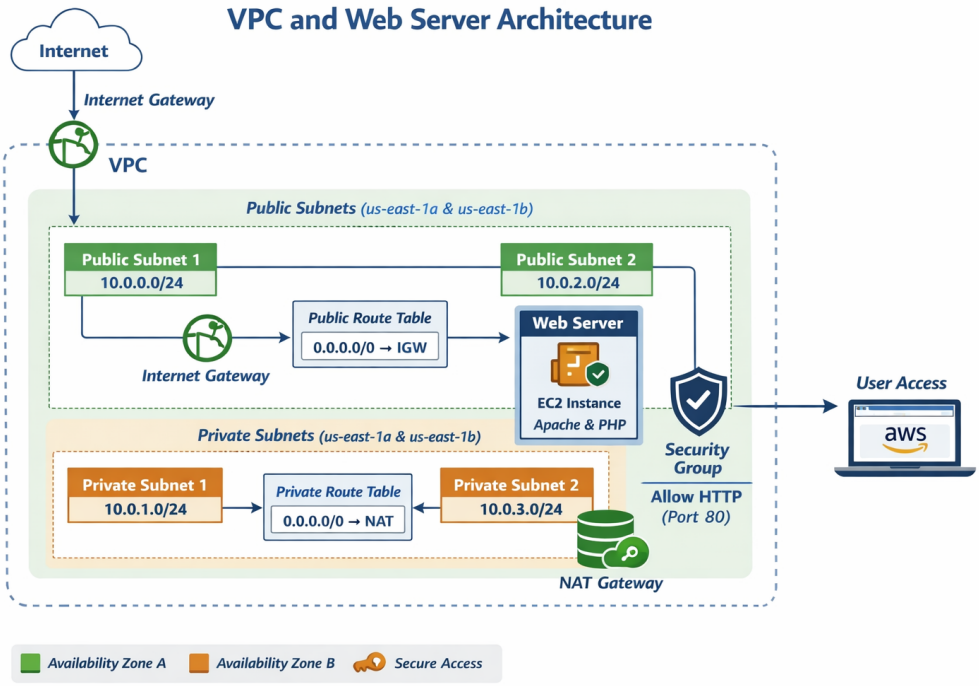

Figure 1: VPC architecture overview – public/private subnets in two AZs, Internet Gateway, NAT Gateway, and web server EC2 in public subnet

Working Methodology

Task 1: Creating the VPC and Initial Resources

I used the VPC Wizard (VPC and more) to create a VPC named "lab-vpc" with CIDR 10.0.0.0/16, one public subnet (10.0.0.0/24) and one private subnet (10.0.1.0/24) in us-east-1a, an Internet Gateway, route tables, and a NAT Gateway in one AZ. I customized the subnet CIDR blocks and enabled DNS hostnames and resolution. **Observations:**

- The wizard automatically created properly named resources and attached the Internet Gateway to the VPC.
- The public subnet's route table routed 0.0.0.0/0 to the Internet Gateway, while the private subnet routed to the NAT Gateway.

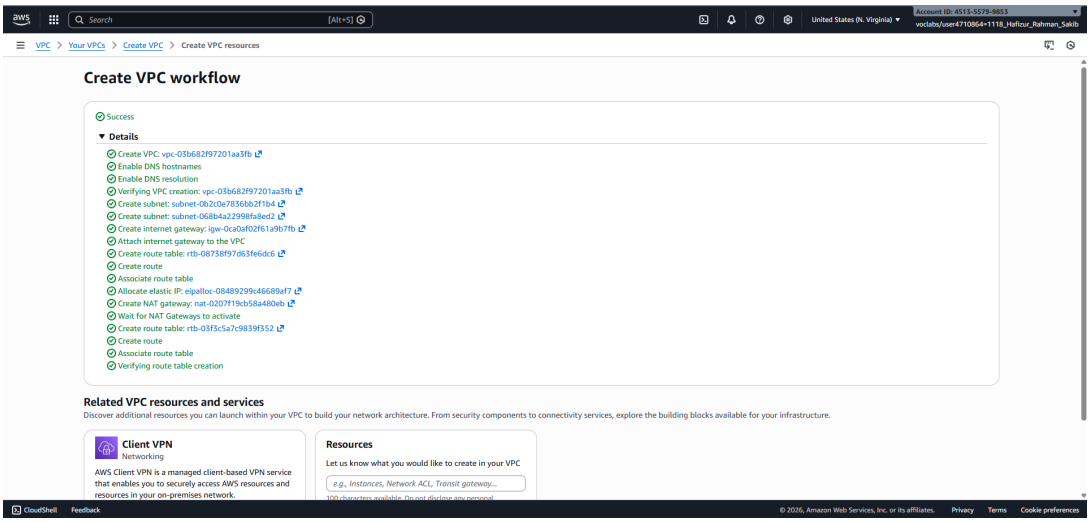

Figure 2: Confirmation of successful VPC creation including lab-vpc, public/private subnets, Internet Gateway, NAT Gateway, and route tables

This screenshot shows the completion message after creating the VPC and associated resources. It confirmed that all components were provisioned correctly with the intended names and CIDR ranges, demonstrating my understanding of automated VPC setup.

Task 2: Creating Additional Subnets and Configuring Route Tables

I created two more subnets in us-east-1b: a public subnet (lab-subnet-public2, 10.0.2.0/24) and a private subnet (lab-subnet-private2, 10.0.3.0/24). Then I associated both new subnets with the existing route tables — public subnets with lab-rtb-public (pointing to Internet Gateway) and private subnets with lab-rtb-private1-us-east-1a (pointing to NAT Gateway). **Completed Actions:**

- Created lab-subnet-public2 and lab-subnet-private2

- Associated both public subnets with the public route table
- Associated both private subnets with the private route table

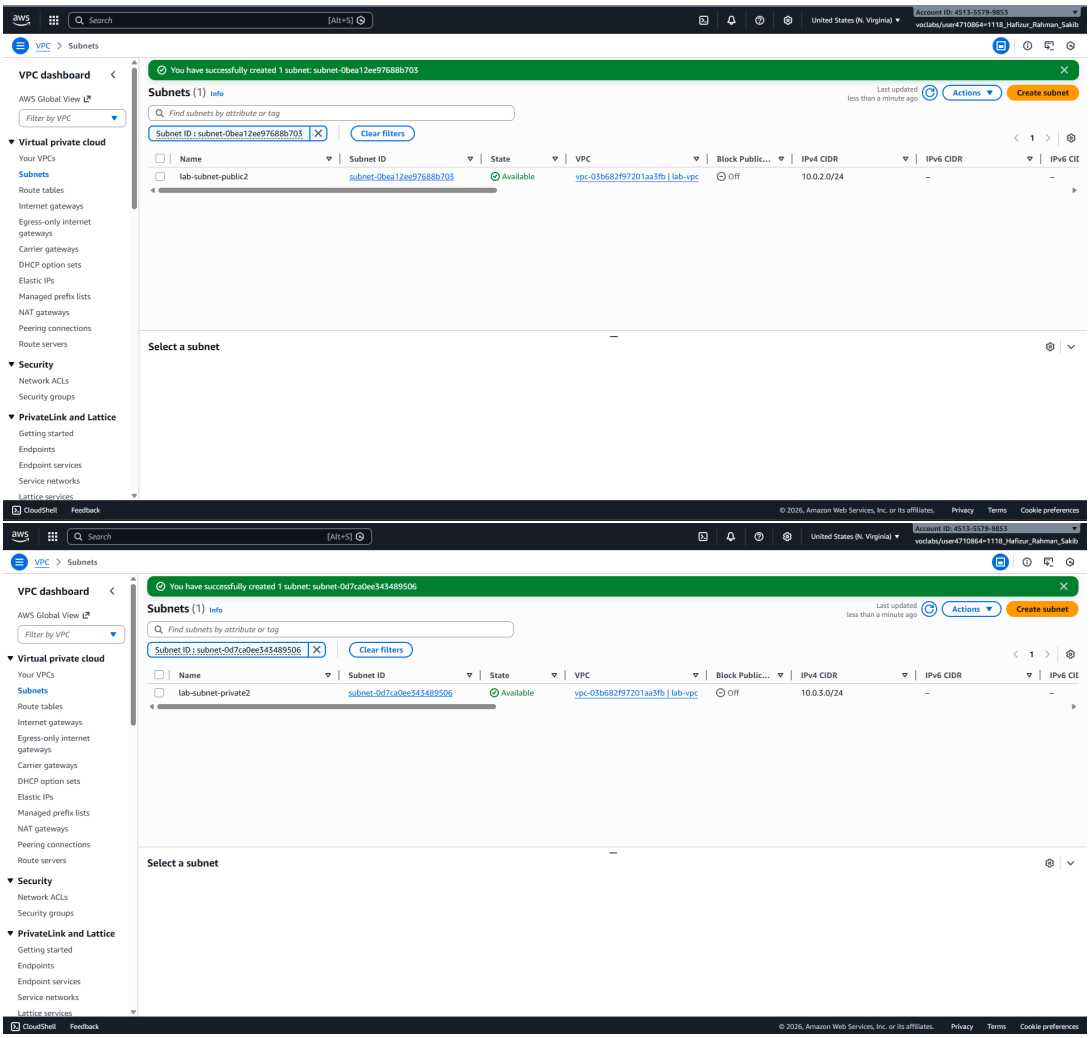

Figure 3: List of all subnets created across two Availability Zones (us-east-1a and us-east-1b)

This screenshot displays all subnets in the VPC console. It verifies that I successfully created subnets in a second AZ for high availability and used correct CIDR blocks without overlap.

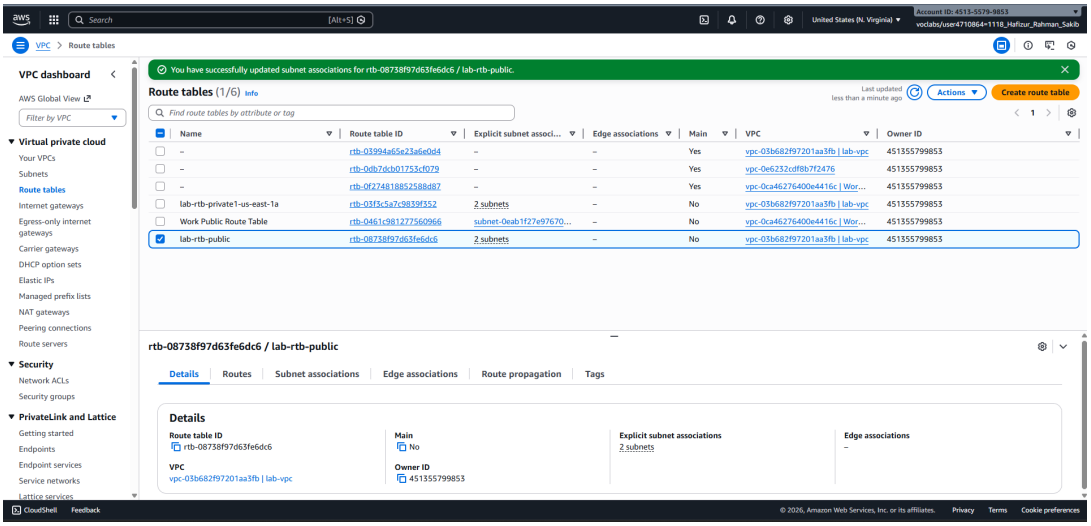

Figure 4: Subnet associations tab showing lab-subnet-public2 correctly associated with the public route table (lab-rtb-public)

This image shows the explicit subnet association edit for the public route table. It demonstrates that I understood how to extend routing configuration to new subnets so that public resources can reach the internet directly.

Task 3: Creating a Security Group

I created a security group named "Web Security Group" in lab-vpc and added an inbound HTTP rule (port 80) from Anywhere-IPv4.

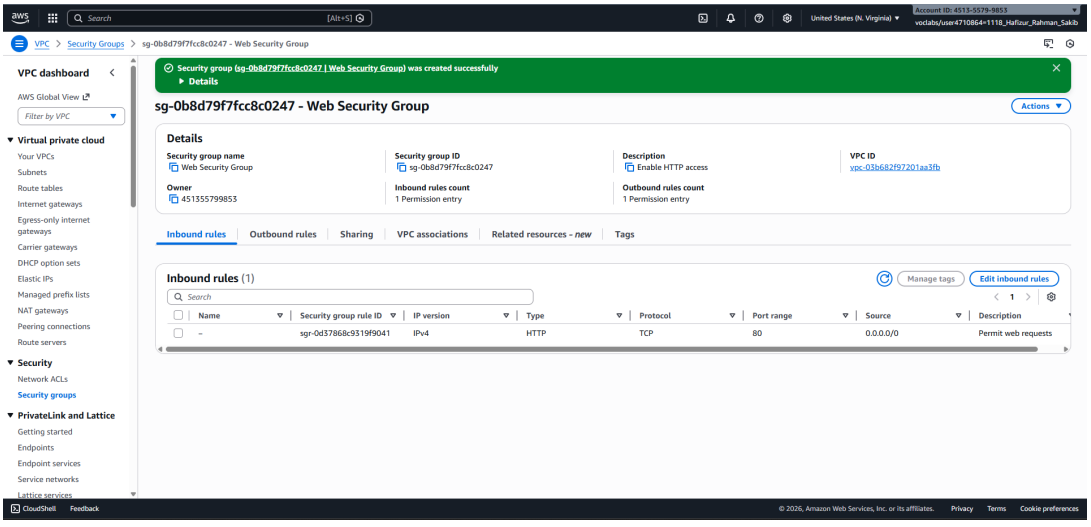

Figure 5: Web Security Group details showing inbound rule allowing HTTP traffic from Anywhere-IPv4

This screenshot confirms the security group was created correctly with only the necessary HTTP inbound rule. It reflects my understanding that security groups act as instance-level firewalls and that HTTP access was required for the web server.

Task 4: Launching and Verifying the Web Server EC2 Instance

I launched an EC2 instance named "Web Server 1" in lab-subnet-public2 (public subnet), using Amazon Linux 2023, t2.micro, vockey key pair, auto-assigned public IP, and the Web Security Group. I pasted the provided user data script to install Apache, PHP, and the sample web application. After launch, I waited for 2/2 status checks to pass, copied the public DNS, and accessed the website in a browser. **Observations:**

- The instance received a public IP and became reachable.
- The user data script executed successfully, serving the sample PHP page with AWS logo and metadata.

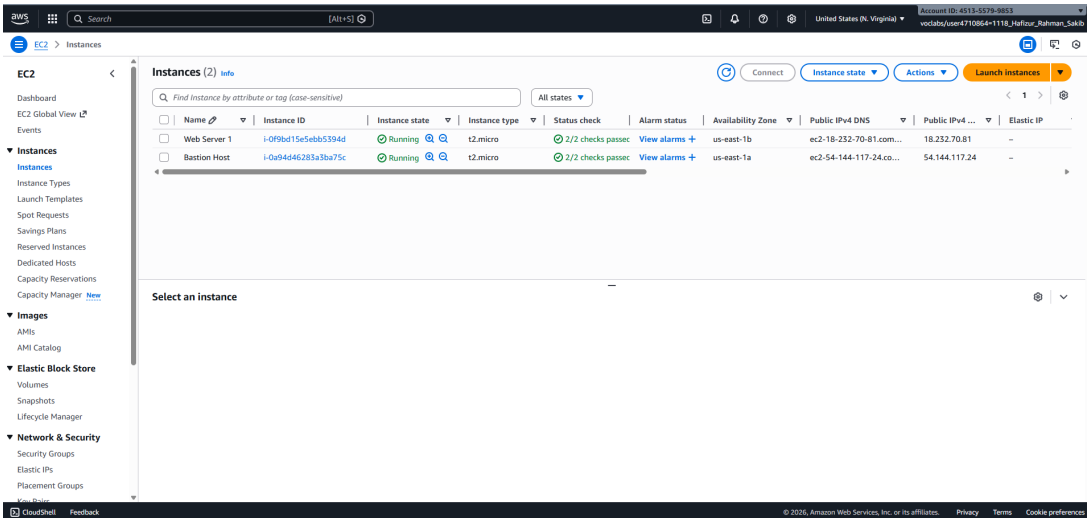

Figure 6: EC2 instance details showing Web Server 1 in running state, public subnet, public IP assigned, and associated with Web Security Group

This screenshot shows the instance summary after launch. It confirms correct placement in a public subnet, security group attachment, and successful status checks.

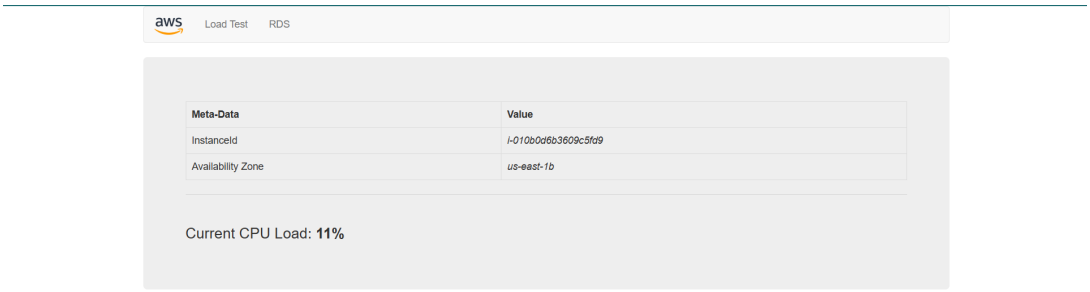

Figure 7: Successful access to the web server showing the sample application page with AWS logo and instance metadata

This final screenshot proves the web server was fully functional and publicly accessible. It validates the entire network configuration — VPC, subnets, routing, security group, and EC2 launch with user data.

Results Summary

All lab tasks were completed successfully and verified through the lab grading system. The final score achieved was **30/30**, confirming correct configuration of the VPC, subnets, route tables, security group, and web server instance.

- Task 1: VPC created correctly
- Task 2a: New subnets created correctly
- Task 2b: Subnet route table association
- Task 3: Security group created correctly
- Task 4a: EC2 instance created correctly
- Task 4b: EC2 instance website accessible

Conclusion

This lab provided me with practical experience in designing and implementing a secure, highly available VPC environment in AWS. I successfully created a VPC spanning two Availability Zones, configured public and private subnets with appropriate internet access, applied least-privilege security group rules, and deployed a functional web server. The exercise reinforced the importance of proper subnet routing, the difference between public and private subnets, and the role of NAT Gateways in maintaining security for private resources. Overall, it strengthened my foundational knowledge of AWS networking, which is critical for building scalable and secure cloud applications.